

EloidaOS

Sistema operacional determinístico baseado em números primos para controle de eventos interdependentes em MCU

1. Visão Geral

O **EloidaOS** é um kernel cooperativo para microcontroladores, projetado especificamente para **controle de eventos** e **correlações matemáticas entre tarefas interdependentes**. Seu objetivo é oferecer **determinismo**, **baixo custo de memória** e **verificabilidade** em sistemas embarcados críticos.

Diferente de um RTOS preemptivo tradicional ou de uma NPU, o EloidaOS ocupa um nicho específico: aplicações onde múltiplos sensores ou eventos precisam ser **correlacionados** para produzir uma decisão de controle **previsível e auditável**.

Princípio fundamental: o tempo é primo, a task é veredito, o MMC é correlação, o desvio é política.

2. Os Dois Modos de Uso

O EloidaOS pode ser utilizado em **dois modos distintos**, conforme a complexidade da aplicação. O modo básico é suficiente para a maioria dos casos; o modo com interdependência é indicado quando tarefas precisam ser correlacionadas.

2.1 Modo Básico — Sem Interdependência

Neste modo, cada tarefa é **independente**. O kernel apenas garante que cada uma execute no intervalo correto, definido pelo seu número primo.

ELEMENTO	FUNÇÃO
RTC (1 Hz)	Base de tempo global

ELEMENTO	FUNÇÃO
Primos	Período de cada tarefa em segundos
Contadores <code>u32</code>	Contagem até o estouro do primo
Tabela de despacho	Execução da tarefa correspondente

Quando usar: aplicações onde as tarefas não dependem umas das outras — leitura de sensores independentes, logs periódicos, blink de LED, heartbeat, etc.

2.2 Modo com Interdependência — MMC e Desvio

Neste modo, cada tarefa **retorna seu próprio número primo** (ou `0` em falha). O **MMC** soma os retornos, produzindo um **número único** que identifica exatamente quais tarefas estão OK. Esse número é usado para **arbitrar** qual tarefa deve assumir o controle, por meio de uma **tabela de desvio**.

ELEMENTO	FUNÇÃO
Retorno primo	Cada task retorna seu primo (OK) ou 0 (falha)
MMC	Soma ponderada dos retornos — correlação matemática
Soma única	Identifica o conjunto de tasks OK
Tabela de desvio	Mapeia soma → task a executar
ISRs / estados	Podem alterar o MMC dinamicamente

Vantagem: a soma de primos distintos é única para cada combinação de tarefas OK. Isso permite decodificar o estado do sistema a partir de um único número.

3. Arquitetura em Camadas

O EloidaOS é organizado em cinco camadas conceituais, que podem ser usadas de forma incremental:

CAMADA	NOME	RESPONSABILIDADE
0	Base de Tempo	RTC, contadores, primos — define <i>quando</i> cada task é candidata
1	Execução	Tabela de despacho, flags <code>READY</code> , <code>run</code> / <code>destroy</code>
2	Veredito	Retorno <code>u8</code> por task, flags <code>DONE</code> / <code>OK</code>
3	Correlação (MMC)	Soma ponderada dos vereditos — número único
4	Arbitragem	Tabela de desvio — mapeia soma → task a executar

As camadas 0 e 1 compõem o **modo básico**. As camadas 2, 3 e 4 adicionam **interdependência e correlação matemática**.

4. Base de Tempo — Primos e RTC

O EluidaOS usa o **RTC com interrupção de 1 Hz** como base de tempo. Cada tarefa tem um contador de 32 bits que é incrementado a cada segundo.

Quando o contador atinge o **número primo** associado à tarefa, a flag `READY` é setada e o contador é zerado.

```
void RTC_IRQHandler(void)
{
    u8 i;
    for (i = 0; i < MAX_TASKS; i++) {
        if (++COUNTS[i] >= PRIME_NUMS[i]) {
            COUNTS[i] = 0;
            READY[i] = 1;
        }
    }
}
```

Por que primos?

- **Intervalos coprimos** — tarefas raramente coincidem no mesmo segundo
- **Distribuição harmônica** — tarefas críticas com primos pequenos, tarefas de fundo com primos grandes
- **Determinismo total** — o comportamento é previsível e inspecionável

- **Overflow em 136 anos** — contadores `u32` a 1 Hz

5. Modo Básico — Exemplo Conceitual

No modo básico, o `main` apenas executa cada tarefa quando sua flag está setada:

```
void main(void)
{
    u8 i;
    while (1) {
        for (i = 0; i < MAX_TASKS; i++) {
            if (READY[i]) {
                TASK_TABLE[i]();
                READY[i] = 0;
            }
        }
    }
}
```

Cada tarefa é **independente**. Não há correlação entre elas. O sistema executa cada uma no seu ritmo, sem se importar com o resultado das demais.

6. Modo com Interdependência — MMC e Desvio

6.1 Retorno por Primo

Cada tarefa retorna **seu próprio número primo** em caso de sucesso, ou `0` em caso de falha:

```
u8 task0(void) {
    if (leitura_invalida) return 0;
    SHARED.tensao_mv = ler_tensao();
    return PRIME_NUMS[0];    /* retorna 2 */
}
```

6.2 MMC — Soma Ponderada

O MMC soma os retornos das tarefas correlacionadas. Como primos distintos produzem somas únicas, o resultado **identifica exatamente** quais tarefas estão OK:

```

u32 mmc(void)
{
    return OK[0] + OK[1] + OK[2] + OK[3] + OK[4] + OK[5] + OK[6];
}

```

6.3 Tabela de Desvio — Arbitragem

A soma MMC é usada como índice em uma tabela que define qual tarefa deve assumir:

```

u8 DESVIO(u32 soma)
{
    if (soma == 0)                return 7; /* failsafe */
    if (soma == (2 + 3 + 5))      return 3; /* calcula SOC */
    if (soma == (2 + 3 + 5 + 7))  return 4; /* balanceia */
    if (soma == (2 + 3 + 5 + 7 + 11 + 13 + 17)) return 6; /* supervisor */
    return 6;
}

```

6.4 Main com Interdependência

```

void main(void)
{
    u8 i, proxima;
    u32 soma;

    while (1)
    {
        /* (a) executa tasks periodicas */
        for (i = 0; i < MAX_TASKS; i++) {
            if (READY[i]) {
                OK[i] = TASK_TABLE[i]();
                DONE[i] = 1;
                READY[i] = 0;
            }
        }

        /* (b) correlaciona via MMC */
        soma = mmc();

        /* (c) consulta tabela de desvio */
        proxima = DESVIO(soma);

        /* (d) executa a task arbitrada */
        OK[proxima] = TASK_TABLE[proxima]();
        DONE[proxima] = 1;
    }
}

```

```
}  
}
```

7. Propriedade Matemática — Unicidade da Soma

A soma de primos distintos é **única** para cada combinação. Isso permite que o MMC produza um **identificador de estado** a partir da soma:

SOMA	CONJUNTO DE TASKS OK	ESTADO DO SISTEMA
0	nenhuma	Failsafe
2	task0	Só tensão OK
5	task2	Só temperatura OK
10	task0, task1, task2	Sensores básicos OK
17	+ task3	SOC calculado
28	+ task4	Balanceamento OK
58	+ task5, task6	Sistema pleno

Resultado: um único número inteiro codifica o estado completo do subsistema correlacionado. A tabela de desvio traduz esse número em ação de controle.

8. Por que Só Faz Sentido com Interdependência

O modo com MMC e desvio só é vantajoso quando:

- As tarefas são **interdependentes** (uma depende do resultado de outra)
- Há **eventos concorrentes** que precisam ser arbitrados
- A decisão precisa ser **determinística** e **auditável**
- O custo precisa ser **mínimo** (memória e CPU)

Se as tarefas fossem independentes, um super loop simples bastaria. O MMC e a tabela de desvio só agregam valor quando há **correlação** entre os resultados.

9. Comparação com Outras Abordagens

ABORDAGEM	DETERMINISMO	MEMÓRIA	VERIFICABILIDADE	NICHO
Super loop	Nenhum	Mínima	Baixa	Aplicações triviais
RTOS preemptivo	Médio	Alta (TCBs, pilhas)	Média	Multitarefa genérica
NPU / rede neural	Aproximado	Grande	Baixa	Percepção, visão, áudio
ElويداOS	Alto	Mínima	Alta	Eventos interdependentes

10. Aplicações Típicas

APLICAÇÃO	VARIÁVEIS CORRELACIONADAS
BMS (Battery Management)	Tensão, corrente, temperatura, SOC
Motor industrial	Encoder, corrente, temperatura, posição
Drone	IMU, GPS, barômetro, magnetômetro
Sistema médico	Pressão, vazão, bolha, dose
Automotivo (ECU)	MAF, lambda, temperatura, injeção
Energia solar	Tensão, corrente, irradiância, MPPT
Automação predial	Temperatura, umidade, CO ₂ , conforto
Voo	Atitude, altitude, velocidade, navegação

Em todos esses casos, há **múltiplos sensores interdependentes, decisão de controle baseada em correlação, requisito de determinismo e restrição de memória.**

11. Princípios do ElويداOS

1. **Tempo é primo** — intervalos determinísticos e coprimos
2. **Task é veredito** — cada tarefa atesta seu resultado
3. **MMC é correlação** — os vereditos são combinados matematicamente
4. **Desvio é política** — a decisão é explícita em tabela
5. **Tudo é estático** — sem heap, sem alocação dinâmica
6. **Tudo é verificável** — a política é inspecionável
7. **Tudo é mínimo** — o kernel cabe em centenas de bytes

12. Considerações Finais

O EloidaOS não é um RTOS genérico, não é uma NPU e não é uma bala de prata. É um **kernel especializado** para um **nicho específico**: controle de eventos interdependentes com correlação matemática determinística.

Pode ser usado em **dois modos**:

- **Modo básico** — apenas distribuição temporal de tarefas independentes
- **Modo com interdependência** — correlação matemática via MMC e arbitragem via tabela de desvio

Em uma frase: o EloidaOS transforma um MCU comum em um controlador determinístico de eventos interdependentes, com custo mínimo de memória e verificabilidade total da política de controle.